WS2.3 – Secure Software Development

Future Factory Applicatie

Groep: 10 – Firewall Force
Groepsleden: Bavli, Jonas, Lucas, Vo

Inleiding

Voor workshop 3 hebben wij de applicatie **Future Factory** gekozen als casus. Dit is een eerder gerealiseerde Laravel-applicatie waarin gewerkt wordt met verschillende gebruikersrollen, voertuigen, modules, planningen en klantgegevens. De workshop vraagt om vier onderdelen uit te werken: de CRA, een architecture review, een security evaluatie en een developer security training. Daarnaast moet het resultaat worden vastgelegd en verwerkt worden in een presentatie voor de terugkoppeling.

Omdat de applicatie voor deze opdracht niet live getest kon worden, is deze uitwerking gebaseerd op de beschikbare code, projectstructuur, logische architectuur en de bekende functionaliteiten van Future Factory. Daardoor is dit document vooral een analyse van de software-opzet en de verwachte securitymaatregelen, en geen technische penetratietest.

Future Factory is een Laravel-applicatie die gebruikmaakt van een **`.env`-bestand** voor omgevingsconfiguratie. In dit bestand worden instellingen opgeslagen zoals de databaseverbinding, applicatie-instellingen en andere gevoelige configuratiegegevens. De functionele werking van de applicatie verloopt verder via Laravel-routes, controllers, modellen en database-interacties. Gebruikers voeren acties uit in de browser, waarna Laravel deze verzoeken verwerkt en gegevens ophaalt of opslaat in de database.

Future Factory is een geschikte casus voor deze workshop, omdat de applicatie voldoende onderdelen bevat om security goed te kunnen beoordelen. Er is sprake van authenticatie, rollen en rechten, gegevensinvoer via formulieren, CRUD-functionaliteit, onderliggende database-interactie en beheer van planningen en modules. Juist dit soort onderdelen maken het mogelijk om de software te beoordelen vanuit secure-by-design, architecture review, threat modeling en secure coding practices.

OPDRACHT 1: CYBER RESILIENCE ACT (CRA)

Auditteam

Leg in je eigen woorden uit wat de CRA inhoudt

De **Cyber Resilience Act (CRA)** is Europese wetgeving die ervoor moet zorgen dat digitale producten en software veiliger worden ontwikkeld en beter onderhouden worden. Het idee achter de CRA is dat security niet pas achteraf opgelost moet worden, maar al vanaf het ontwerp moet worden meegenomen. Een product moet dus niet alleen functioneel zijn, maar ook veilig genoeg ontworpen, gebouwd en beheerd worden.

Voor software betekent dit concreet dat ontwikkelaars rekening moeten houden met veilige configuratie, het beperken van kwetsbaarheden, het goed omgaan met updates en patching, en

het beschermen van gebruikers en gegevens tegen misbruik. Security moet dus onderdeel zijn van de levenscyclus van de software, en niet slechts een optionele extra achteraf.

Binnen Future Factory betekent dit dat de applicatie niet alleen moet werken voor planners, monteurs, klanten en inkopers, maar ook veilig moet omgaan met rollen, inloggegevens, invoer, foutmeldingen en gevoelige projectinstellingen. Een applicatie die rollen gebruikt, formulieren verwerkt en gegevens opslaat, moet namelijk aantoonbaar aandacht besteden aan vertrouwelijkheid, integriteit en beschikbaarheid.

Realiseer een vragenlijst om te toetsen of de software van het developmentteam voldoet aan de CRA en of hun software veilig is

Om Future Factory te toetsen aan de gedachte achter de CRA, hebben wij onderstaande auditvragen opgesteld.

Auditvragenlijst CRA

1. Is security vanaf het ontwerp meegenomen in de applicatie?
2. Is er authenticatie aanwezig voor gebruikers?
3. Werkt de applicatie met rollen en toegangsbeperkingen?
4. Wordt invoer aan de serverzijde gevalideerd?
5. Worden gevoelige gegevens veilig opgeslagen of afgeschermd?
6. Is foutinformatie voldoende beperkt voor eindgebruikers?
7. Worden configuratie en secrets veilig beheerd?
8. Is er aandacht voor dependencybeheer en kwetsbaarheden in packages?
9. Is er logging of monitoring voor securityrelevante acties?
10. Is de applicatie onderhoudbaar vanuit security-oogpunt, bijvoorbeeld met updates en patching?

Developmentteam

Kies met elkaar een software applicatie die je eerder hebt gerealiseerd

Voor deze opdracht hebben wij gekozen voor **Future Factory**. Dit is een eerder gerealiseerde Laravel-applicatie die gebruikt wordt als casus voor deze workshop. De applicatie is gebouwd rondom het beheren van voertuigen, modules, planningen en rollen binnen een productiecontext. Hoewel de applicatie niet door de hele groep samen is gebouwd, hebben wij deze als gezamenlijke analysecasus gebruikt.

Bereid de audit voor door de architectuur, ontwerpen en software code op orde te brengen

Voor de audit hebben wij eerst de applicatie op hoofdlijnen in kaart gebracht. Future Factory is opgebouwd als Laravel-webapplicatie en maakt gebruik van een `.env`**-bestand** voor de omgevingsconfiguratie. In dit bestand staan instellingen zoals databasegegevens, applicatieconfiguratie en andere gevoelige omgevingsvariabelen. Dat is een gebruikelijke en

goede werkwijze, zolang dit bestand niet openbaar wordt gedeeld of in versiebeheer terechtkomt.

De functionele opbouw van de applicatie bestaat verder uit een webinterface voor verschillende gebruikersrollen, Laravel-routes, controllers voor de verwerking van gebruikersacties, modellen voor de gegevenslogica en een database waarin informatie over gebruikers, voertuigen, modules en planningen wordt opgeslagen.

Een gebruiker voert dus een actie uit in de browser, Laravel verwerkt deze aanvraag via routes en controllers, en via de modellen worden gegevens in de database opgehaald of opgeslagen. Juist deze combinatie van configuratie via `.env` en logica via routes, controllers en modellen maakt het mogelijk om de applicatie vanuit securityperspectief te beoordelen.

Evalueer of de software applicatie in deze staat veilig is

Future Factory laat zien dat er al verschillende basismaatregelen aanwezig zijn die bijdragen aan een veiliger ontwerp. De applicatie werkt met gebruikersrollen, waardoor niet iedere gebruiker overal toegang toe hoeft te hebben. Ook is het logisch dat Laravel-validatie wordt gebruikt voor formulieren en CRUD-acties, wat positief is voor het beperken van foutieve of kwaadwillige invoer.

Tegelijkertijd beoordelen wij de applicatie in deze staat als **gedeeltelijk veilig, maar nog niet volwassen veilig**. De software bevat wel een basis van securitymaatregelen, maar mist naar verwachting nog verdere hardening, expliciete security requirements, structurele logging en sterk ingerichte defense-in-depth. Ook veilige omgang met configuratie en secrets is een belangrijk aandachtspunt. Daarmee is de applicatie bruikbaar en niet direct onveilig, maar nog niet op het niveau dat je van een volledig secure-by-design systeem zou verwachten.

Welke secure-by-design principes jullie vooraf beter hadden moeten meenemen

Als wij Future Factory opnieuw zouden ontwerpen, zouden wij vooraf sterker rekening houden met de volgende secure-by-design principes:

- security requirements al in de ontwerpfase vastleggen
- threat modeling uitvoeren voordat functionaliteiten gebouwd worden
- attack surface analysis maken van alle invoer- en uitvoerpunten
- secure-by-default instellingen gebruiken
- autorisatie op meerdere lagen afdwingen
- logging en monitoring vanaf het begin meenemen
- secrets en configuratie gescheiden en veilig beheren

Gezamenlijk

Voer de audit uit met behulp van de vragenlijst en leg het resultaat en conclusie vast

Op basis van de vragenlijst beoordelen wij Future Factory als volgt.

Auditresultaat CRA

1. Is security vanaf het ontwerp meegenomen?
Beoordeling: Gedeeltelijk
Er zijn aanwijzingen dat security deels is meegenomen, bijvoorbeeld via rollen, login en validatie. Toch lijkt security nog niet volledig als expliciet ontwerpprincipe te zijn vastgelegd.

2. Is er authenticatie aanwezig voor gebruikers?
Beoordeling: Ja
De applicatie werkt met ingelogde gebruikers en meerdere rollen.

3. Werkt de applicatie met rollen en toegangsbeperkingen?
Beoordeling: Ja
Dit is één van de sterkere punten van de applicatie.

4. Wordt invoer aan serverzijde gevalideerd?
Beoordeling: Gedeeltelijk tot Ja
Het is aannemelijk dat Laravel-validatie wordt gebruikt, maar niet alle validatie is expliciet als securitymaatregel gedocumenteerd.

5. Worden gevoelige gegevens veilig opgeslagen of afgeschermd?
Beoordeling: Gedeeltelijk
Wachtwoorden worden binnen Laravel normaal gesproken veilig opgeslagen. Tegelijkertijd is het belangrijk dat gevoelige configuratiegegevens, zoals database-instellingen en applicatiesleutels, goed beschermd blijven. Omdat Future Factory gebruikmaakt van een `.env`-bestand, is veilige omgang met dit bestand een belangrijk aandachtspunt.

6. Is foutinformatie voldoende beperkt voor eindgebruikers?
Beoordeling: Gedeeltelijk
In een ontwikkelapplicatie is dit vaak nog onvoldoende afgeschermd.

7. Worden configuratie en secrets veilig beheerd?
Beoordeling: Gedeeltelijk
De applicatie gebruikt een `.env`-bestand voor omgevingsinstellingen en secrets. Dat is op zichzelf positief, omdat gevoelige gegevens daarmee buiten de gewone code gehouden kunnen worden. Wel blijft het essentieel dat dit bestand niet in een publieke repository belandt en alleen toegankelijk is voor bevoegde personen. Dit zien wij als een belangrijk securitypunt binnen de applicatie.

8. Is er aandacht voor dependencybeheer en kwetsbaarheden in packages?
Beoordeling: Gedeeltelijk
Omdat Laravel externe packages gebruikt, moet hier blijvend aandacht voor zijn.

9. Is er logging of monitoring voor securityrelevante acties?
Beoordeling: Nee of onvoldoende zichtbaar
Dit lijkt nog geen sterk uitgewerkt onderdeel.

10. Is de applicatie onderhoudbaar vanuit security-oogpunt?
Beoordeling: Gedeeltelijk
De basis is er, maar securityonderhoud en structurele patching zijn nog niet duidelijk uitgewerkt.

Conclusie opdracht 1

Onze conclusie is dat Future Factory een aantal goede basismaatregelen bevat die passen bij de gedachte achter de CRA, zoals authenticatie, rolverdeling en validatie. Toch is de applicatie nog niet volledig uitgewerkt volgens een volwassen secure software development aanpak. Vooral op het gebied van logging, secrets management, expliciete security requirements en structureel onderhoud zijn nog verbeteringen nodig. Daardoor voldoet de applicatie deels aan de richting van de CRA, maar nog niet volledig aan het gewenste volwassenheidsniveau.

Maak hiervan een paar slides voor de presentatie tijdens de terugkoppeling

Voor de presentatie zouden wij bij opdracht 1 de volgende slides maken:

- korte uitleg van de CRA
- gekozen applicatie: Future Factory
- auditvragenlijst in verkorte vorm
- auditresultaten met Ja / Gedeeltelijk / Nee
- conclusie en belangrijkste verbeterpunten

OPDRACHT 2: ARCHITECTURE REVIEW & DESIGN REQUIREMENTS

Kies met elkaar een software applicatie die je eerder hebt gerealiseerd

Voor deze opdracht gebruiken wij opnieuw **Future Factory** als casus. Deze applicatie bevat voldoende functionaliteiten en meerdere rollen, waardoor de architectuur en de designkeuzes goed beoordeeld kunnen worden.

Beschrijf de 3 belangrijkste design requirements die je in een nieuw project altijd zou meenemen om security by design te waarborgen

1. Alleen geauthenticeerde gebruikers mogen gevoelige functionaliteiten gebruiken

Binnen een applicatie als Future Factory mogen functies zoals voertuigen beheren, planningen aanpassen en modules bewerken nooit bereikbaar zijn voor anonieme bezoekers. Iedere gevoelige actie moet dus achter authenticatie zitten.

2. Autorisatie moet per rol en per actie worden afgedwongen

Een gebruiker mag alleen die functionaliteiten gebruiken die passen bij zijn of haar rol. Een klant mag bijvoorbeeld niet dezelfde handelingen kunnen uitvoeren als een planner of inkoper. Daarom moet autorisatie niet alleen op pagina- of routeniveau gebeuren, maar ook op actieniveau.

3. Alle invoer moet server-side gevalideerd worden

Formulieren en requests moeten altijd aan serverzijde gecontroleerd worden op type, formaat, grenzen en logische regels. Frontend-validatie alleen is nooit voldoende.

Maak een architectuur-review van jullie huidige software en identificeer ten minste 2 designkeuzes die de security verbeteren en 2 die risico's opleveren. Motiveer met onderbouwing waarom een keuze veilig of onveilig is

Architectuur-review van Future Factory

De globale architectuur van Future Factory bestaat uit:

- een browser of webclient
- Laravel-routes
- controllers voor de afhandeling van aanvragen
- modellen voor de gegevensverwerking
- een database voor opslag
- een `.env`-bestand voor omgevingsconfiguratie

Hierbij is het belangrijk om onderscheid te maken tussen **configuratie** en **applicatielogica**. Het `.env`-bestand regelt de omgevingsinstellingen van de applicatie, zoals databaseverbindingen en andere gevoelige configuratiegegevens. De echte functionele werking van de applicatie verloopt via de routes, controllers, modellen en database.

Binnen deze architectuur werken meerdere rollen, zoals planner, monteur, klant en inkoper. Dat betekent dat er meerdere security-aandachtspunten zijn, bijvoorbeeld bij authenticatie, autorisatie, validatie en veilige omgang met configuratiegegevens.

Twee designkeuzes die de security verbeteren

1. Rolgebaseerde toegang

Het gebruik van meerdere rollen is een sterke designkeuze. Hiermee wordt voorkomen dat iedere gebruiker bij alle functionaliteiten kan. Dit ondersteunt het least privilege-principe en verlaagt het risico op ongeautoriseerde toegang.

2. Server-side verwerking in Laravel

Het feit dat logica, validatie en gegevensverwerking centraal via de backend plaatsvinden, is veiliger dan wanneer belangrijke controles alleen in de frontend zouden zitten. Hierdoor kunnen beveiligingsmaatregelen consistenter worden afgedwongen.

Twee designkeuzes die risico's opleveren

1. Mogelijke te sterke afhankelijkheid van routebeveiliging

Wanneer autorisatie vooral op routeniveau wordt afgedwongen en minder expliciet in de achterliggende businesslogica, kan dat risico's geven. Een veilige applicatie moet op meerdere lagen controleren of een gebruiker iets mag doen.

2. Beperkte zichtbaarheid van logging en monitoring

Het ontwerp lijkt vooral gericht op functionaliteit, rollen en validatie, maar minder op detectie en monitoring. Daardoor is het lastiger om misbruik, fouten of verdachte acties te signaleren en later te onderzoeken.

Conclusie opdracht 2

De architectuur van Future Factory heeft een duidelijke en bruikbare basis. Vooral het gebruik van rollen en de backendstructuur van Laravel helpen om security beter te organiseren. Tegelijkertijd zou de applicatie sterker worden wanneer autorisatie nog explicieter op meerdere lagen wordt afgedwongen en logging en monitoring vanaf het ontwerp meegenomen worden.

Maak van het resultaat een aantal slides voor de presentatie tijdens de terugkoppeling

Voor de presentatie zouden wij bij opdracht 2 de volgende slides maken:

- 3 belangrijkste design requirements
- simpele architectuurweergave van Future Factory
- 2 veilige designkeuzes
- 2 risicovolle designkeuzes
- korte conclusie

OPDRACHT 3: SECURITY EVALUATIE

Deze opdracht is wat technischer en moet zo uitgewerkt worden dat ook niet-technische studenten mee kunnen denken over de risico's en maatregelen.

Kies met elkaar een software applicatie die je eerder hebt gerealiseerd

Voor deze opdracht gebruiken wij opnieuw **Future Factory**.

Voer een attack surface analyse uit: maak een diagram van alle invoer/uitvoerpunten en bepaal waar een aanvaller zou kunnen binnenkomen

Attack surface analyse

Het aanvalsoppervlak van Future Factory bestaat uit alle punten waar gegevens, opdrachten of gebruikersacties de applicatie binnenkomen of verlaten. Voor deze applicatie zijn de belangrijkste invoer- en uitvoerpunten:

- loginfunctionaliteit
- gebruikersrollen en dashboardtoegang
- formulieren voor voertuigen
- formulieren voor modules
- formulieren voor planning
- profiel- of accountfunctionaliteit
- databaseacties achter CRUD-operaties
- foutmeldingen en responses
- eventuele uploads of afbeeldingvelden

Tekstueel attack surface diagram

Anonieme gebruiker / ingelogde gebruiker
→ browser
→ Laravel-routes
→ controllers
→ validatie en autorisatie
→ modellen
→ database
→ response terug naar gebruiker

Daarnaast speelt ook de **omgevingsconfiguratie via het `.env`-bestand** een belangrijke rol in de security van de applicatie. Dit bestand is geen invoerpunt voor gewone gebruikers, maar bevat wel gevoelige configuratiegegevens die goed beschermd moeten worden. Wanneer zulke gegevens uitlekken, kan dat grote gevolgen hebben voor de veiligheid van de applicatie.

Belangrijke assets zijn:

- gebruikersaccounts
- rolrechten
- voertuiggegevens
- planningen
- module-informatie
- klantgegevens
- configuratie en secrets uit het `.env`-bestand

Maak een threat model en beschrijf 3–5 realistische dreigingen

Threat model Future Factory

Voor Future Factory hebben wij vier realistische dreigingen geselecteerd.

1. Ongeautoriseerde toegang tot functionaliteit van andere rollen

Een gebruiker probeert toegang te krijgen tot functies die eigenlijk bedoeld zijn voor een andere rol, bijvoorbeeld via een directe URL, aangepaste request of onjuiste autorisatiecontrole.

Impact: hoog
Kans: middel

2. Manipulatie van formulierinvoer

Een gebruiker voert expres onjuiste of aangepaste waarden in om gegevens te vervalsen of logische regels te omzeilen.

Impact: middel tot hoog
Kans: hoog

3. Brute-force op login

Een aanvaller probeert wachtwoorden te raden of hergebruikt gelekte inloggegevens om een account over te nemen.

Impact: hoog
Kans: middel

4. Information disclosure via foutmeldingen of configuratie

Een aanvaller krijgt te veel informatie over de applicatie door debug-output, technische foutmeldingen of onveilige omgang met configuratiebestanden zoals het `.env`-bestand.

Impact: hoog
Kans: middel

Geef per dreiging aan welke beveiligingsmaatregel jullie zouden implementeren of al hebben geïmplementeerd. Lever zowel een ontwerp als een beargumenteerde toelichting van deze maatregelen

Maatregelen per dreiging

Dreiging 1 – Ongeautoriseerde toegang

Maatregel: autorisatie op meerdere lagen afdwingen, bijvoorbeeld via middleware, controllerchecks en policies.
Toelichting: alleen routeniveau is niet altijd genoeg. Door defense-in-depth toe te passen, wordt het moeilijker om rechten te omzeilen.

Dreiging 2 – Manipulatie van formulierinvoer

Maatregel: server-side validatie op alle formulieren en extra controle op business rules.
Toelichting: een Laravel-applicatie met veel formulieren moet niet alleen controleren of data technisch geldig is, maar ook logisch toegestaan is.

Dreiging 3 – Brute-force op login

Maatregel: rate limiting, tijdelijke blokkering na meerdere mislukte pogingen en sterk wachtwoordbeleid.
Toelichting: login is een logisch aanvalsdoelwit. Deze maatregelen verlagen de kans op geautomatiseerde aanvallen.

Dreiging 4 – Information disclosure

Maatregel: debug uitschakelen in productie, generieke foutmeldingen gebruiken en het `.env`-bestand strikt buiten gedeelde of openbare omgevingen houden.
Toelichting: gebruikers mogen geen technische detailinformatie krijgen die aanvallers helpt om de applicatie beter te begrijpen. Daarnaast mogen gevoelige configuratiegegevens, zoals database-inloggegevens en applicatiesleutels, nooit uitlekken.

Wat valt je op in de samenwerking tussen technische en niet-technische studenten?

Bij deze opdracht valt op dat technische en niet-technische studenten verschillende sterke kanten hebben. Technische studenten kijken sneller naar routes, validatie, controllers, modellen en databaserelaties. Niet-technische studenten kijken juist duidelijker naar gebruikersperspectief, impact, begrijpelijkheid en procesrisico's. Juist die combinatie is sterk, omdat security niet alleen om code gaat, maar ook om gedrag, gebruik en gevolgen in de praktijk.

Conclusie opdracht 3

De grootste risico's in Future Factory liggen bij autorisatie, invoer, login en informatielekken. Door deze risico's expliciet te benoemen en per dreiging een maatregel te koppelen, wordt duidelijk waar de software versterkt kan worden.

Maak van het resultaat een aantal slides voor de presentatie tijdens de terugkoppeling

Voor de presentatie zouden wij bij opdracht 3 de volgende slides maken:

- attack surface diagram
- 4 belangrijkste dreigingen
- maatregel per dreiging
- observatie over samenwerking technisch / niet-technisch
- conclusie

OPDRACHT 4: DEVELOPER SECURITY TRAINING

Deze opdracht is wat minder technisch en vraagt om een mini-trainingsmodule die past bij de eigen softwarecontext, dus niet om een generieke lijst.

Kies met elkaar een software applicatie die je eerder hebt gerealiseerd

Voor deze opdracht gebruiken wij opnieuw **Future Factory**.

Stel een voorstel van een ontwerp van een mini-trainingsmodule (max. 1 A4) op die toekomstige developers van jullie team zouden moeten volgen om veilig(er) te programmeren. Welke onderwerpen worden behandeld en waarom?

Voorstel mini-trainingsmodule

Titel

Mini Security Training voor Future Factory Developers

Doel

Nieuwe developers leren hoe zij veilig werken binnen de Future Factory-applicatie, zodat nieuwe functionaliteiten niet alleen werkend maar ook veilig worden opgeleverd.

Doelgroep

Nieuwe developers die aan Future Factory gaan werken.

Onderwerpen en waarom

1. Authenticatie en autorisatie

Developers moeten begrijpen dat inloggen alleen niet genoeg is. Elke gevoelige functie moet gekoppeld zijn aan de juiste rol en rechten.
Waarom: de applicatie werkt met meerdere rollen, waardoor rechtenbeheer essentieel is.

2. Server-side validatie

Alle invoer moet aan serverzijde worden gecontroleerd.
Waarom: de applicatie bevat veel formulieren voor voertuigen, modules en planningen.

3. Veilig omgaan met configuratie en secrets

Databasegegevens, app keys en andere gevoelige instellingen worden binnen Laravel meestal beheerd via het `.env`-bestand. Developers moeten weten dat dit bestand alleen bedoeld is voor omgevingsconfiguratie en niet openbaar gedeeld of in versiebeheer geplaatst mag worden.
Waarom: verkeerd omgaan met het `.env`-bestand kan direct leiden tot uitlek van gevoelige informatie en daarmee tot ernstige securityproblemen.

4. Veilige foutafhandeling

Gebruikers mogen geen technische details zien die intern of voor aanvallers bruikbaar zijn.
Waarom: foutmeldingen kunnen informatie lekken over de architectuur of configuratie.

5. Dependencybeheer en updates

Externe packages moeten gecontroleerd en bijgewerkt worden.
Waarom: kwetsbaarheden in packages kunnen ook de veiligheid van de applicatie aantasten.

6. Logging en securitybewust code reviewen

Belangrijke acties moeten gelogd worden en code reviews moeten ook naar security kijken.
Waarom: security moet een vast onderdeel van het teamproces worden.

Welke secure coding practices moeten verplicht aan bod komen?

Voor Future Factory moeten de volgende secure coding practices verplicht behandeld worden:

- auth op alle gevoelige routes
- autorisatie per rol en per actie
- server-side validatie op alle input
- geen hardcoded secrets
- veilige foutmeldingen
- dependencycontrole en patching
- security meenemen in code reviews

Hoe zorgen jullie dat het toepasbaar en meetbaar is in jullie project?

De training wordt toepasbaar gemaakt door deze direct te koppelen aan het ontwikkelproces. Dat betekent:

- elke nieuwe route krijgt auth- en autorisatiecontrole
- elke nieuwe form submit krijgt validatieregels
- elke pull request krijgt een korte security-check
- secrets worden buiten gedeelde code gehouden
- dependencybeheer wordt periodiek gecontroleerd

Meetbaar maken wij dit door de volgende criteria te gebruiken:

- 100% van de gevoelige routes is afgeschermd
- 100% van de formulieren heeft server-side validatie
- 0 hardcoded secrets in de codebase
- elke pull request bevat een securitycheck

- dependencycontrole gebeurt periodiek
- nieuwe developers volgen de training bij onboarding

Eis: de module moet op maat zijn voor jullie softwarecontext, niet een generieke lijst

Deze trainingsmodule is specifiek afgestemd op Future Factory. De gekozen onderwerpen passen direct bij de aard van de applicatie: een Laravel-systeem met meerdere rollen, formulieren, CRUD-functionaliteit, gevoelige configuratie en gegevensverwerking. Daardoor is dit geen algemene securitylijst, maar een projectgerichte security-aanpak.

Conclusie opdracht 4

De developer security training helpt om security niet als losse theorie te behandelen, maar als praktisch onderdeel van het dagelijkse ontwikkelwerk. Door de training toe te spitsen op rollen, validatie, secrets, foutafhandeling en dependencybeheer sluit deze goed aan op de risico's van Future Factory.

Maak van het resultaat een aantal slides voor de presentatie tijdens de terugkoppeling

Voor de presentatie zouden wij bij opdracht 4 de volgende slides maken:

- mini-trainingsmodule samengevat
- verplichte secure coding practices
- toepasbaarheid binnen Future Factory
- meetbare criteria
- conclusie

Eindconclusie

Op basis van alle vier de opdrachten concluderen wij dat Future Factory een bruikbare basis heeft voor veilige softwareontwikkeling, maar nog niet het niveau bereikt van een volledig volwassen secure-by-design applicatie. De applicatie bevat al sterke elementen, zoals authenticatie, rolverdeling en server-side verwerking. Tegelijkertijd zijn er duidelijke verbeterpunten op het gebied van expliciete security requirements, logging, secrets management, defense-in-depth en structureel securityonderhoud.

De workshop laat zien dat veilige software niet alleen draait om het voorkomen van bugs, maar om een bredere aanpak waarin CRA, architecture review, threat modeling, attack surface analysis en security training samenkomen. Future Factory is daardoor een goede casus om te laten zien waar de basis al goed is en waar verdere professionalisering nodig is.

Reflectie

Door deze workshop is duidelijk geworden dat security niet iets is wat je pas aan het einde toevoegt. Veel risico's ontstaan al bij ontwerpkeuzes, rechtenstructuren, invoerpunten en projectafspraken. Ook blijkt dat een applicatie die functioneel goed werkt, nog steeds tekort kan schieten op securityvlak wanneer logging, onderhoud, configuratie en secure-by-design niet voldoende zijn meegenomen.

Wat wij vooral meenemen uit deze workshop, is dat veilige softwareontwikkeling een combinatie is van techniek, proces en bewustzijn. Juist daarom zijn architecture review, threat modeling, attack surface analysis en developer training zulke belangrijke onderdelen van een SSDLC.